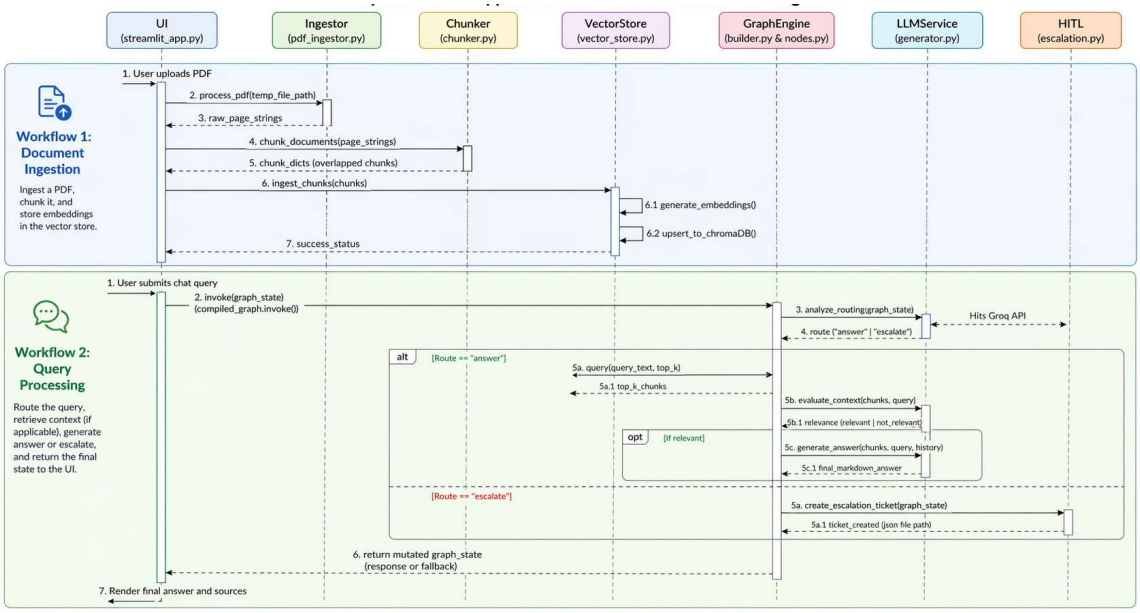
Low-Level Design (LLD): RAG Support Assistant

1. Module-Level Design



Document Processing Module: Implemented via 'app/rag/pdf_ingestor.py', extracting text natively from raw PDFs using PyPDF handlers to sweep individual pages and build memory payloads prior to chunking.

Chunking Module: Located in 'app/rag/chunker.py'. Employs recursive overlapping splits (1000 chunk size, 200 overlap) to generate context-rich tokens, preventing abrupt sentence cut-offs.

Embedding Module: Implemented in 'app/rag/embeddings.py'. Orchestrates HuggingFace local models (specifically all-MiniLM-L6-v2) for in-memory tensor mappings without API rate limits.

Vector Storage Module: Utilizes a custom 'VectorStoreManager' wrapped around ChromaDB's PersistentClient. Sandboxes data structurally by mapping unique Web-session IDs directly to distinct collections.

Retrieval Module: Located in 'app/rag/retriever.py'. Translates incoming textual queries into vectors using the same instantiated embedding function and polls ChromaDB for the Top-8 dense equivalents via cosine distance/L2.

Query Processing Module: Exposed via 'app/llm/generator.py'. Encapsulates prompt construction mapping incoming raw text against rigid zero-shot classification constraints, parsing JSON configurations seamlessly.

Graph Execution Module: Located in 'app/graph/nodes.py' and 'app/graph/builder.py'. Transforms typical linear chains into a Stateful Graph. Functions map natively to distinct Nodes passing TypedDict payloads.

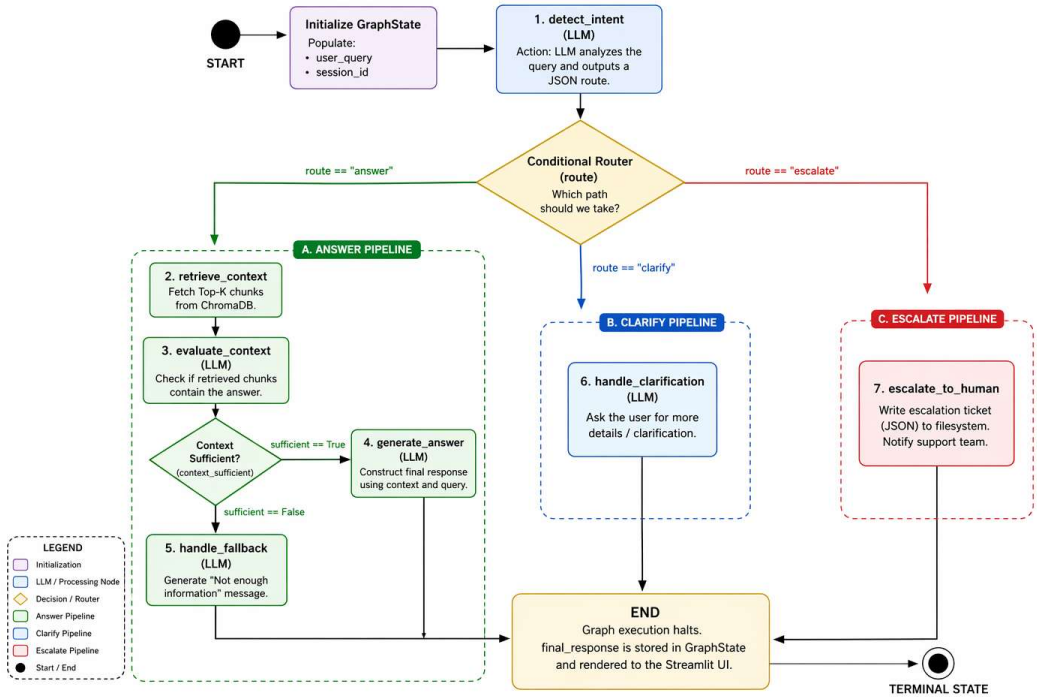
HITL Module: Contained within 'app/hitl/escalation.py' and 'reviewer.py'. Serializes support tickets natively to the filesystem (JSON) preserving the exact state for Reviewer retrieval from Streamlit sidebar endpoints.

2. Data Structures

- Document Representation: Raw dictionaries containing pure string text paired with page/source metadata bounds.
- Chunk Format: Extends document representation. Schema: {'text': string, 'metadata': {'source': string, 'page': int, 'chunk_id': int}}.
- Embedding Structure: High-dimensional fixed-float arrays (384-dimensional standard via MiniLM) assigned string representation IDs representing exact chunk origins.
- Query-Response Schema: JSON string payloads returned organically by the routing LLM. e.g. {'intent': 'clarify', 'route': 'clarify'}.

State Object for Graph: A TypedDict instance (GraphState) passed securely between edges. Fields:
- session_id (str)
- user_query (str)
- uploaded_files (list)
- retrieved_chunks (list)
- sources (list)
- intent (str)
- route (str)
- context_sufficient (bool)
- answer_draft (str)
- final_response (str)
- escalation_required (bool)

3. Workflow Design (LangGraph)



- Nodes (Processing/Output):

  • detect_intent -> Orchestrates LLM JSON Routing.
  • retrieve_context -> Pulls matching vectorized chunks.
  • evaluate_context -> Check if chunks sufficiently answer the query.
  • generate_answer -> Constructs response and cites sources.
  • handle_clarification / handle_fallback / escalate_to_human -> Terminal states solving distinct path branches.

- Edges & Data Flow:

The START edge immediately flows to `detect_intent`. Conditional edge maps the `route` variable to either `retrieve_context` (if route='answer'), `handle_clarification` (if route='clarify'), or `escalate_to_human` (if route='escalate'). The data flowing continuously updates the mutable properties inside the active `GraphState` TypedDict.

4. Conditional Routing Logic

- Answer Generation Criteria: Routing prompt verifies the query asks a legitimate question targeting uploaded content or explicitly asking for a document summary.
- Escalation Criteria: Engaged passively if users explicitly display emotional frustration, state 'talk to an agent', or reference complex external billing/compliance scopes outside generic queries.

- Low Confidence / Missing Context: Evaluated explicitly natively after Retrieval blocks. If the LLM Context Evaluator identifies a failure, `state['context_sufficient']` becomes False, rerouting the edge instantly to `handle_fallback`.
- Complex Query: Processed linearly as an Answer query but evaluated heavily against maximum chunk overlaps.

5. HITL Design

- Trigger Conditions: Invoked exclusively when the Router isolates severe keywords / intense confusion states mapping the execution Node securely to 'escalate_to_human'.
- Post-Escalation Lifecycle: The system modifies a localized JSON ticket ledger mapping the active session ID, user intent, and query context payload. The graph exits generating a placeholder response: 'Your request has been escalated'.
- Human Response Integration: The Streamlit Sidebar exposes a 'Reviewer Dashboard' natively scraping the JSON ledger. The administrator resolves tickets concurrently without halting global session access or impacting parallel user chains.

6. API / Interface Design

- Input Format: Uses graphical drag-and-drop mechanics to temporarily serialize PDFs into Streamlit sessions, joined universally by simple Text-Input Chat components polling user queries natively as strings.
- Output Format: Multi-dimensional markdown arrays. Generates standard block-responses appended with expandable 'View Sources' contextual dropdowns containing exact cited chunks for rigorous auditability.
- Interaction Flow: Upload -> Spinner Parsing -> Ingestion Success Notification -> Chat Query -> Graph Spinners ('Analyzing and Routing') -> Final Markdown Render + Expandable Debug Meta.

7. Error Handling

- Missing Data: Trapped automatically if the Graph is evaluated without indexed collections; gracefully invokes the fallback method explaining sufficient data isn't exposed.
- No Relevant Chunks Found: If ChromaDB Euclidean queries surface random outliers, the distinct `evaluate_context` Node acts as an air-gap constraint. Evaluated explicitly natively, throwing False if LLM refuses context relevance mapping it seamlessly into local failures without alerting clients to backend system crashes.
- LLM Failure: All LLM integrations (like Router parse outputs) use try-except handlers standardizing corrupted responses (like missing JSON boundaries) into generic structured routing values ('intent': 'unknown', 'route': 'answer') preventing fatal cascading Graph outages.